

Principal Component Analysis and Autoencoders

Shuiwang Ji

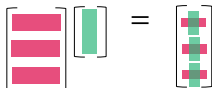
Department of Computer Science & Engineering
Texas A&M University

The Art of Linear Algebra - Matrix times vector¹

Mv

Matrix times Vector – 2 Ways

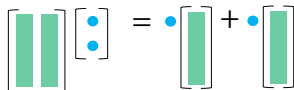
Mv1



The row vectors of A are multiplied by a vector $\mathbf{x}$ and become the three dot-product elements of $A\mathbf{x}$.

$$A\mathbf{x} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} (x_1 + 2x_2) \\ (3x_1 + 4x_2) \\ (5x_1 + 6x_2) \end{bmatrix}$$

Mv2



The product $A\mathbf{x}$ is a linear combination of the column vectors of A .

$$A\mathbf{x} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1 \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} + x_2 \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

At first, you learn (Mv1). But when you get used to viewing it as (Mv2), you can understand $A\mathbf{x}$ as a linear combination of the columns of A . Those products fill the column space of A denoted as $\mathbf{C}(A)$. The solution space of $A\mathbf{x} = \mathbf{0}$ is the nullspace of A denoted as $\mathbf{N}(A)$.

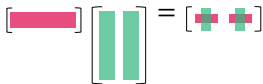
¹<https://github.com/kenjihiranabe/The-Art-of-Linear-Algebra/>

The Art of Linear Algebra - Vector times matrix

vM

Vector times Matrix – 2 Ways

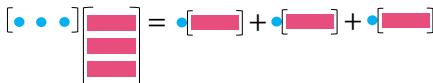
vM1


$$\begin{bmatrix} \text{pink} \end{bmatrix} \begin{bmatrix} \text{green} & \text{green} \end{bmatrix} = \begin{bmatrix} \text{green} & \text{green} \end{bmatrix}$$

$$\mathbf{y}A = \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = \begin{bmatrix} (y_1 + 3y_2 + 5y_3) & (2y_1 + 4y_2 + 6y_3) \end{bmatrix}$$

A row vector $\mathbf{y}$ is multiplied by the two column vectors of A and become the two dot-product elements of $\mathbf{y}A$.

vM2


$$\begin{bmatrix} \text{blue} & \text{blue} & \text{blue} \end{bmatrix} \begin{bmatrix} \text{pink} \\ \text{pink} \\ \text{pink} \end{bmatrix} = \text{blue} \begin{bmatrix} \text{pink} \end{bmatrix} + \text{blue} \begin{bmatrix} \text{pink} \end{bmatrix} + \text{blue} \begin{bmatrix} \text{pink} \end{bmatrix}$$

$$\mathbf{y}A = \begin{bmatrix} y_1 & y_2 & y_3 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} = y_1 \begin{bmatrix} 1 & 2 \end{bmatrix} + y_2 \begin{bmatrix} 3 & 4 \end{bmatrix} + y_3 \begin{bmatrix} 5 & 6 \end{bmatrix}$$

The product $\mathbf{y}A$ is a linear combination of the row vectors of A .

The Art of Linear Algebra - Matrix times matrix

MM

Matrix times Matrix – 4 Ways

MM
1

$$\begin{bmatrix} \text{pink} \\ \text{pink} \\ \text{pink} \end{bmatrix} \begin{bmatrix} \text{green} & \text{green} \end{bmatrix} = \begin{bmatrix} \text{green} & \text{green} \\ \text{green} & \text{green} \\ \text{green} & \text{green} \end{bmatrix}$$

Every element becomes a dot product of row vector and column vector.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = \begin{bmatrix} (x_1+2x_2) & (y_1+2y_2) \\ (3x_1+4x_2) & (3y_1+4y_2) \\ (5x_1+6x_2) & (5y_1+6y_2) \end{bmatrix}$$

MM
2

$$\begin{bmatrix} \text{gray} \end{bmatrix} \begin{bmatrix} \text{green} & \text{green} \end{bmatrix} = \begin{bmatrix} \text{gray} & \text{green} & \text{gray} & \text{green} \end{bmatrix} = \begin{bmatrix} \text{green} & \text{green} \\ \text{green} & \text{green} \end{bmatrix}$$

Ax and Ay are linear combinations of columns of A .

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = A \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = [Ax \quad Ay]$$

MM
3

$$\begin{bmatrix} \text{pink} \\ \text{pink} \\ \text{pink} \end{bmatrix} \begin{bmatrix} \text{gray} \end{bmatrix} = \begin{bmatrix} \text{pink} & \text{gray} \\ \text{pink} & \text{gray} \\ \text{pink} & \text{gray} \end{bmatrix} = \begin{bmatrix} \text{pink} & \text{gray} \\ \text{pink} & \text{gray} \\ \text{pink} & \text{gray} \end{bmatrix}$$

The produced rows are linear combinations of rows.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \end{bmatrix} = \begin{bmatrix} a_1^* \\ a_2^* \\ a_3^* \end{bmatrix} X = \begin{bmatrix} a_1^* X \\ a_2^* X \\ a_3^* X \end{bmatrix}$$

MM
4

$$\begin{bmatrix} \text{green} & \text{green} \end{bmatrix} \begin{bmatrix} \text{pink} \\ \text{pink} \end{bmatrix} = \begin{bmatrix} \text{pink} & \text{gray} \\ \text{green} & \text{gray} \end{bmatrix} + \begin{bmatrix} \text{pink} & \text{gray} \\ \text{green} & \text{gray} \end{bmatrix}$$

Multiplication AB is broken down to a sum of rank 1 matrices.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_1 & a_2 \end{bmatrix} \begin{bmatrix} b_1^* \\ b_2^* \end{bmatrix} = a_1 b_1^* + a_2 b_2^* \\ = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \end{bmatrix} + \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} \begin{bmatrix} b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} \\ 3b_{11} & 3b_{12} \\ 5b_{11} & 5b_{12} \end{bmatrix} + \begin{bmatrix} 2b_{21} & 2b_{22} \\ 4b_{21} & 4b_{22} \\ 6b_{21} & 6b_{22} \end{bmatrix}$$

Practical Patterns (2/3)

P1'


$$\begin{bmatrix} \text{green bar} & \text{green bar} & \text{green bar} \end{bmatrix} \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} = \begin{bmatrix} \text{green bar} & \text{green bar} & \text{green bar} \end{bmatrix}$$

Applying a diagonal matrix from the right scales each column.

$$AD = \begin{bmatrix} \mathbf{a}_1 & \mathbf{a}_2 & \mathbf{a}_3 \end{bmatrix} \begin{bmatrix} d_1 & & \\ & d_2 & \\ & & d_3 \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{a}_1 & d_2 \mathbf{a}_2 & d_3 \mathbf{a}_3 \end{bmatrix}$$

P2'


$$\begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{pink bar} \\ \text{pink bar} \\ \text{pink bar} \end{bmatrix} = \begin{bmatrix} \bullet \text{ pink bar} \\ \bullet \text{ pink bar} \\ \bullet \text{ pink bar} \end{bmatrix}$$

Applying a diagonal matrix from the left scales each row.

$$DB = \begin{bmatrix} d_1 & & \\ & d_2 & \\ & & d_3 \end{bmatrix} \begin{bmatrix} \mathbf{b}_1^* \\ \mathbf{b}_2^* \\ \mathbf{b}_3^* \end{bmatrix} = \begin{bmatrix} d_1 \mathbf{b}_1^* \\ d_2 \mathbf{b}_2^* \\ d_3 \mathbf{b}_3^* \end{bmatrix}$$

Burn this into your memories and you can see ...

Practical Patterns (3/3)

P3

$$\begin{bmatrix} \text{green bar} & \text{green bar} & \text{green bar} \end{bmatrix} \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} = \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{green bar} \end{bmatrix} + \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{green bar} \end{bmatrix} + \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{green bar} \end{bmatrix}$$

This pattern makes another combination of columns.
You will encounter this in differential/recurrence equations.

$$XD\mathbf{c} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \mathbf{x}_3 \end{bmatrix} \begin{bmatrix} d_1 & & \\ & d_2 & \\ & & d_3 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = c_1 d_1 \mathbf{x}_1 + c_2 d_2 \mathbf{x}_2 + c_3 d_3 \mathbf{x}_3$$

P4

$$\begin{bmatrix} \text{green bar} & \text{green bar} & \text{green bar} \end{bmatrix} \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} = \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{green bar} \end{bmatrix} + \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{green bar} \end{bmatrix} + \begin{bmatrix} \bullet & & \\ & \bullet & \\ & & \bullet \end{bmatrix} \begin{bmatrix} \text{green bar} \end{bmatrix}$$

A matrix is broken down to a sum of rank 1 matrices,
as in singular value/eigenvalue decomposition.

$$U\Sigma V^T = \begin{bmatrix} \mathbf{u}_1 & \mathbf{u}_2 & \mathbf{u}_3 \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \sigma_3 \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \mathbf{v}_3^T \end{bmatrix} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \sigma_3 \mathbf{u}_3 \mathbf{v}_3^T$$

Orthogonal Matrices

- ① An orthogonal matrix is a square matrix whose columns and rows are orthogonal unit vectors, *i.e.*, orthonormal vectors. That is, if a matrix Q is an orthogonal matrix, we have

$$Q^T Q = Q Q^T = I.$$

- ② It leads to $Q^{-1} = Q^T$, which is a very useful property as it provides an easy way to compute the inverse.
- ③ For an orthogonal $n \times n$ matrix $Q = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n]$, where $\mathbf{q}_i \in \mathbb{R}^n$, $i = 1, 2, \dots, n$, it is easy to see that $\mathbf{q}_i^T \mathbf{q}_j = 0$ when $i \neq j$ and $\mathbf{q}_i^T \mathbf{q}_i = 1$.
- ④ Furthermore, suppose $Q_1 = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_i]$ and $Q_2 = [\mathbf{q}_{i+1}, \mathbf{q}_{i+2}, \dots, \mathbf{q}_n]$, we have $Q_1^T Q_1 = I$, $Q_2^T Q_2 = I$, but $Q_1 Q_1^T \neq I$, $Q_2 Q_2^T \neq I$.

- ❶ A square $n \times n$ matrix $\mathbf{S}$ with n linearly independent eigenvectors can be factorized as

$$\mathbf{S} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{-1},$$

where $\mathbf{Q}$ is the square $n \times n$ matrix whose columns are eigenvectors of $\mathbf{S}$, and $\mathbf{\Lambda}$ is the diagonal matrix whose diagonal elements are the corresponding eigenvalues.

- ❷ Note that only diagonalizable matrices can be factorized in this way.
- ❸ If $\mathbf{S}$ is a symmetric matrix, its eigenvectors are orthogonal. Thus $\mathbf{Q}$ is an orthogonal matrix and we have

$$\mathbf{S} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T.$$

The Art of Linear Algebra - Eigen-Decomposition

$$S = Q\Lambda Q^T$$

Eigenvalue decomposition of a symmetric matrix S

All the eigenvalues are real, and all the eigenvectors can be chosen orthonormal.

$$\begin{array}{c} S \\ \left[\begin{array}{|c|} \hline \square \\ \hline \end{array} \right] \end{array} = \begin{array}{c} Q \\ \left[\begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline \end{array} \right] \end{array} \begin{array}{c} \Lambda \\ \left[\begin{array}{|c|} \hline \bullet \\ \hline \end{array} \right] \end{array} \begin{array}{c} Q^T \\ \left[\begin{array}{|c|} \hline 1 \\ \hline \end{array} \right] \end{array} = \begin{array}{c} \lambda_1 \mathbf{q}_1 \mathbf{q}_1^T \\ \left[\begin{array}{|c|} \hline 1 \\ \hline \end{array} \right] \end{array} + \begin{array}{c} \lambda_2 \mathbf{q}_2 \mathbf{q}_2^T \\ \left[\begin{array}{|c|} \hline 2 \\ \hline \end{array} \right] \end{array} + \begin{array}{c} \lambda_3 \mathbf{q}_3 \mathbf{q}_3^T \\ \left[\begin{array}{|c|} \hline 3 \\ \hline \end{array} \right] \end{array} \quad \text{using P4}$$

A symmetric matrix S is diagonalized into Λ by an orthogonal matrix Q and its transpose.

And it is broken down into a sum of rank 1 projection matrices (known as the spectrum theory).

$$S = Q\Lambda Q^T = \begin{bmatrix} | & | & | \\ \mathbf{q}_1 & \mathbf{q}_2 & \mathbf{q}_3 \\ | & | & | \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \end{bmatrix} \begin{bmatrix} -\mathbf{q}_1^T & - \\ -\mathbf{q}_2^T & - \\ -\mathbf{q}_3^T & - \end{bmatrix} = \lambda_1 \begin{bmatrix} | \\ \mathbf{q}_1 \\ | \end{bmatrix} \begin{bmatrix} - & \mathbf{q}_1^T & - \end{bmatrix} + \lambda_2 \begin{bmatrix} | \\ \mathbf{q}_2 \\ | \end{bmatrix} \begin{bmatrix} - & \mathbf{q}_2^T & - \end{bmatrix} + \lambda_3 \begin{bmatrix} | \\ \mathbf{q}_3 \\ | \end{bmatrix} \begin{bmatrix} - & \mathbf{q}_3^T & - \end{bmatrix}$$

$$= \lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3$$

$$S = S^T$$

$$Q^T Q = P_1 + P_2 + P_3 = I$$

$$P_1 P_2 = P_2 P_3 = P_3 P_1 = 0$$

The Art of Linear Algebra - Singular Value Decomposition

$$A = U\Sigma V^T$$

All matrices including rectangular ones can be singular value decomposed (SVD).

$$A = U \Sigma V^T = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T$$

using P4

You can find V as an orthonormal basis of $\mathbb{R}^n$, and U as an orthonormal basis of $\mathbb{R}^m$ together they diagonalize A into Σ . This is the Singular Value Decomposition. And it is also also a sum of rank 1 matrices.

$$A = U\Sigma V^T = \begin{bmatrix} | & | & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \mathbf{u}_3 \\ | & | & | \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \end{bmatrix} \begin{bmatrix} -\mathbf{v}_1^T & - \\ -\mathbf{v}_2^T & - \end{bmatrix} = \sigma_1 \begin{bmatrix} | \\ \mathbf{u}_1 \\ | \end{bmatrix} [-\mathbf{v}_1^T -] + \sigma_2 \begin{bmatrix} | \\ \mathbf{u}_2 \\ | \end{bmatrix} [-\mathbf{v}_2^T -]$$

$$= \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T$$

$$\begin{aligned} UU^T &= I_m \\ VV^T &= I_n \end{aligned}$$

Singular Value Decomposition

The singular value decomposition (SVD) of an $m \times n$ **real** matrix (without loss of generality, we assume $m \geq n$) can be written as

$$\mathbf{R} = \mathbf{U} \tilde{\Sigma} \mathbf{V}^T,$$

where $\mathbf{U}$ is an orthogonal $m \times m$ matrix, $\mathbf{V}$ is an orthogonal $n \times n$ matrix, and $\tilde{\Sigma}$ is a diagonal $m \times n$ matrix with non-negative real values on diagonal. That is,

$$\begin{aligned} \mathbf{U}^T \mathbf{U} &= \mathbf{U} \mathbf{U}^T = \mathbf{I}_{m \times m}, \\ \mathbf{V}^T \mathbf{V} &= \mathbf{V} \mathbf{V}^T = \mathbf{I}_{n \times n}, \\ \tilde{\Sigma} &= \begin{bmatrix} \Sigma_{n \times n} \\ \mathbf{0} \end{bmatrix}_{m \times n}, \Sigma_{n \times n} = \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ 0 & 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \dots & \sigma_n \end{bmatrix}, \end{aligned} \quad (1)$$

where $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ are known as singular values. If $\text{rank}(\mathbf{R}) = r$ ($r \leq n$), we have $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ and $\sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_n = 0$.

Relation to Eigen-Decomposition

The columns of $\mathbf{U}$ (left-singular vectors) are orthonormal eigenvectors of $\mathbf{R}\mathbf{R}^T$, and the columns of $\mathbf{V}$ (right-singular vectors) are orthonormal eigenvectors of $\mathbf{R}^T\mathbf{R}$. In other words, we have

$$\mathbf{R}\mathbf{R}^T = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1},$$

$$\mathbf{R}^T\mathbf{R} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}.$$

It is easy to verify them as we have

$$\mathbf{R}^T\mathbf{R} = (\mathbf{U} \begin{bmatrix} \mathbf{\Sigma} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^T)^T \mathbf{U} \begin{bmatrix} \mathbf{\Sigma} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^T = \mathbf{V}([\mathbf{\Sigma} \quad \mathbf{0}] \begin{bmatrix} \mathbf{\Sigma} \\ \mathbf{0} \end{bmatrix}) \mathbf{V}^T = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T,$$

$$\mathbf{R}\mathbf{R}^T = \mathbf{U} \begin{bmatrix} \mathbf{\Sigma} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^T (\mathbf{U} \begin{bmatrix} \mathbf{\Sigma} \\ \mathbf{0} \end{bmatrix} \mathbf{V}^T)^T = \mathbf{U}(\begin{bmatrix} \mathbf{\Sigma} \\ \mathbf{0} \end{bmatrix} [\mathbf{\Sigma} \quad \mathbf{0}]) \mathbf{U}^T = \mathbf{U} \begin{bmatrix} \mathbf{\Sigma}^2 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{U}^T$$

and $\mathbf{V}^T = \mathbf{V}^{-1}$, $\mathbf{U}^T = \mathbf{U}^{-1}$.

SVD and eigen-decomposition

- ① Under what conditions are SVD and eigen-decomposition the same? First, $\mathbf{R}$ is a symmetric matrix, i.e., $\mathbf{R} = \mathbf{R}^T$. Second, $\mathbf{R}$ is a positive semi-definite matrix, i.e., $\forall \mathbf{x} \in \mathbb{R}^n, \mathbf{x}^T \mathbf{R} \mathbf{x} \geq 0$.
- ② The difference between $\mathbf{\Lambda}$ in eigen-decomposition and $\mathbf{\Sigma}$ in SVD is that, the diagonal entries of $\mathbf{\Lambda}$ can be negative, while the diagonal entries of $\mathbf{\Sigma}$ are non-negative. What are the fundamental reasons underlying this difference? Why the requirements on the singular values in SVD (non-negative and in sorted order) do not prevent the generality of SVD?

Compact SVD

If $\text{rank}(\mathbf{R}) = r$ ($r \leq n$), we have

$$\mathbf{R} = \mathbf{U} \tilde{\Sigma} \mathbf{V}^T$$

$$= [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_r, \dots, \mathbf{u}_m] \begin{bmatrix} \sigma_1 & \dots & 0 & & & \\ & \ddots & \vdots & & \ddots & \\ 0 & \dots & \sigma_r & & & \\ & & & 0 & \dots & 0 \\ & & & \vdots & \ddots & \vdots \\ & & & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_r^T \\ \vdots \\ \mathbf{v}_n^T \end{bmatrix}.$$

By removing zero components, we obtain

$$\begin{aligned} \mathbf{R} &= \mathbf{U}_r \Sigma_r \mathbf{V}_r^T \\ &= [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_r] \begin{bmatrix} \sigma_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_r \end{bmatrix} \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_r^T \end{bmatrix} \\ &= [\sigma_1 \mathbf{u}_1, \sigma_2 \mathbf{u}_2, \dots, \sigma_r \mathbf{u}_r] \begin{bmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_r^T \end{bmatrix} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T, \end{aligned}$$

where $\text{rank}(\sigma_i \mathbf{u}_i \mathbf{v}_i^T) = 1$, $i = 1, 2, \dots, r$.

Truncated SVD and Best Low-Rank Approximation

We can also approximate the matrix $\mathbf{R}$ with the k largest singular values as

$$\mathbf{R}_k = \mathbf{U}_k \Sigma_k \mathbf{V}_k^T = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

Apparently, $\mathbf{R} \neq \mathbf{R}_k$ unless $\text{rank}(\mathbf{R}) = k$. This approximation is the best in following sense:

$$\begin{aligned} \min_{\mathbf{B}: \text{rank}(\mathbf{B}) \leq k} \|\mathbf{R} - \mathbf{B}\|_F &= \|\mathbf{R} - \mathbf{R}_k\|_F = \sqrt{\sum_{i=k+1}^n \sigma_i^2}, \\ \min_{\mathbf{B}: \text{rank}(\mathbf{B}) \leq k} \|\mathbf{R} - \mathbf{B}\|_2 &= \|\mathbf{R} - \mathbf{R}_k\|_2 = \sigma_{k+1}, \end{aligned}$$

where $\|\cdot\|_F$ denotes the Frobenius norm and $\|\cdot\|_2$ denotes the spectral norm, defined as the largest singular value of the matrix. That is, $\mathbf{R}_k$ is the best rank- k approximation to $\mathbf{R}$ in terms of both the Frobenius norm and spectral norm. Note the difference in terms of approximation errors when different matrix norms are used.

What is PCA?

- ➊ Principal Component Analysis (PCA) is a statistical procedure that can be used to achieve feature (dimensionality) reduction.
- ➋ Note, feature reduction is different from feature selection. After feature reduction, we still use all the features, while feature selection selects a subset of features to use.
- ➌ The goal of PCA is to project the high-dimensional features to a lower-dimensional space with maximal variance and minimum reconstruction error simultaneously.
- ➍ We derive PCA based on maximizing variance, and then we show the solution also minimizes reconstruction error.
- ➎ In machine learning, PCA is an unsupervised learning technique, and therefore does not need labels.

- ➊ To introduce PCA, we start from the simple case where PCA projects the features to a 1-dimensional space.
- ➋ Formally, suppose we have n p -dimensional ($p > 1$) features $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n \in \mathbb{R}^p$.
- ➌ Let $\mathbf{a} \in \mathbb{R}^p$ represent a projection that $\mathbf{a}^T \mathbf{x}_i = z_i$, $i = 1, 2, \dots, n$ where $z_1, z_2, \dots, z_n \in \mathbb{R}^1$.
- ➍ PCA aims to solve

$$\mathbf{a}^* = \arg \max_{\|\mathbf{a}\|=1} \frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2.$$

- ➎ Note that the variance of the reduced data is

$$\frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2,$$

which means that PCA tries to find the projection with the maximum variance in reduced data.

PCA to 1D

Since

$$\bar{z} = \frac{1}{n} \sum_{i=1}^n z_i = \frac{1}{n} \sum_{i=1}^n \mathbf{a}^T \mathbf{x}_i = \mathbf{a}^T \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \right) = \mathbf{a}^T \bar{\mathbf{x}},$$

the problem can be written as

$$\begin{aligned} \mathbf{a}^* &= \arg \max_{\|\mathbf{a}\|=1} \frac{1}{n} \sum_{i=1}^n (z_i - \bar{z})^2 \\ &= \arg \max_{\|\mathbf{a}\|=1} \frac{1}{n} \sum_{i=1}^n (\mathbf{a}^T \mathbf{x}_i - \bar{z})^2 \\ &= \arg \max_{\|\mathbf{a}\|=1} \frac{1}{n} \sum_{i=1}^n (\mathbf{a}^T \mathbf{x}_i - \mathbf{a}^T \bar{\mathbf{x}})^2 \\ &= \arg \max_{\|\mathbf{a}\|=1} \frac{1}{n} \sum_{i=1}^n \mathbf{a}^T (\mathbf{x}_i - \bar{\mathbf{x}}) (\mathbf{x}_i - \bar{\mathbf{x}})^T \mathbf{a} \\ &= \arg \max_{\|\mathbf{a}\|=1} \mathbf{a}^T \underbrace{\left(\frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}}) (\mathbf{x}_i - \bar{\mathbf{x}})^T \right)}_{p \times p \text{ covariance matrix}} \mathbf{a} \\ &= \arg \max_{\|\mathbf{a}\|=1} \mathbf{a}^T \mathbf{C} \mathbf{a}, \end{aligned}$$

where $\mathbf{C} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}}) (\mathbf{x}_i - \bar{\mathbf{x}})^T$, denotes the covariance matrix.

PCA to k -dimensional space

- ❶ What if we want to project the features to a k -dimensional space?
Then the PCA problem becomes

$$\mathbf{A}^* = \arg \max_{\mathbf{A} \in \mathbb{R}^{p \times k} : \mathbf{A}^T \mathbf{A} = \mathbf{I}_k} \text{trace} \left(\mathbf{A}^T \mathbf{C} \mathbf{A} \right), \quad (2)$$

where $\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_k] \in \mathbb{R}^{p \times k}$. Note that when projecting onto k -dimensional space, PCA requires different projection vectors to be orthogonal. Also, the trace above is the sum of the variances after projecting the data to each of the k directions as

$$\text{trace} \left(\mathbf{A}^T \mathbf{C} \mathbf{A} \right) = \sum_{i=1}^k \left(\mathbf{a}_i^T \mathbf{C} \mathbf{a}_i \right).$$

- 1 Solving the problem in Eqn. (2) requires the follow theorem.
- 2 **Theorem.** (*Ky Fan*) Let $\mathbf{H} \in \mathbb{R}^{n \times n}$ be a symmetric matrix with eigenvalues

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n,$$

and the corresponding eigenvectors $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_n]$. Then

$$\lambda_1 + \cdots \lambda_k = \max_{\mathbf{A} \in \mathbb{R}^{n \times k}: \mathbf{A}^T \mathbf{A} = \mathbf{I}_k} \text{trace} \left(\mathbf{A}^T \mathbf{H} \mathbf{A} \right).$$

And the optimal $\mathbf{A}^*$ is given by $\mathbf{A}^* = [\mathbf{u}_1, \dots, \mathbf{u}_k] \mathbf{Q}$ with $\mathbf{Q}$ an arbitrary orthogonal matrix. ■

- ① Note that in Eqn. (2), the covariance matrix $\mathbf{C}$ is a symmetric matrix. Given the above theorem, we directly obtain

$$\begin{aligned}\lambda_1 + \cdots \lambda_k &= \arg \max_{\mathbf{A} \in \mathbb{R}^{n \times k}: \mathbf{A}^T \mathbf{A} = \mathbf{I}_k} \text{trace} \left(\mathbf{A}^T \mathbf{C} \mathbf{A} \right), \\ \mathbf{A}^* &= [\mathbf{u}_1, \dots, \mathbf{u}_k] \mathbf{Q},\end{aligned}$$

where $\lambda_1, \dots, \lambda_k$ are the k largest eigenvalues of the covariance matrix $\mathbf{C}$, and the solution $\mathbf{A}^*$ is the matrix whose columns are corresponding eigenvectors.

- ② It also follows from the above theorem that solutions to PCA are not unique, and they differ by an orthogonal matrix. We used the special case where $\mathbf{Q} = \mathbf{I}$, i.e., $\mathbf{A}^* = [\mathbf{u}_1, \dots, \mathbf{u}_k]$.

How to compute PCA efficiently?

① We define

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \in \mathbb{R}^{p \times n},$$

$$\bar{\mathbf{X}} = \frac{1}{n} \mathbf{X} \mathbf{1}_n \in \mathbb{R}^{p \times 1},$$

$$\tilde{\mathbf{X}} = (\mathbf{X} - \bar{\mathbf{X}} \mathbf{1}_n^T) \in \mathbb{R}^{p \times n},$$

where $\mathbf{1}_n$ is the n -dimensional all-one vector. Here, $\tilde{\mathbf{X}}$ is the centered $\mathbf{X}$, which is obtained by subtracting the mean from each column.

② Then we have

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T = \frac{1}{n} \tilde{\mathbf{X}} \tilde{\mathbf{X}}^T \in \mathbb{R}^{p \times p}.$$

If p is large, it is very costly to compute eigenvectors of $\mathbf{C}$ directly. Moreover, if $p \gg n$, there are known computational problems.

How to compute PCA efficiently?

- ① However, we show that the SVD of $\tilde{\mathbf{X}}$ provides what we need. If the SVD of $\tilde{\mathbf{X}}$ is $\tilde{\mathbf{X}} = \mathbf{U}\tilde{\Sigma}\mathbf{V}^T$, the columns of $\mathbf{U}$ (left-singular vectors) are orthonormal eigenvectors of $\tilde{\mathbf{X}}\tilde{\mathbf{X}}^T$. Note that we have $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$ in Eqn. (1) and thus, the first k columns of $\mathbf{U}$ correspond to the largest k eigenvalues of $\tilde{\mathbf{X}}\tilde{\mathbf{X}}^T$.
- ② With the SVD of $\tilde{\mathbf{X}}$, if we want to project features to a k -dimensional ($k < p$) space, simply take the first k columns of $\mathbf{U}$ as the projection matrix. Generally, the process of computing PCA can be described as

$$\underbrace{\mathbf{X}}_{p \times n} \rightarrow \underbrace{\tilde{\mathbf{X}}}_{p \times n} \rightarrow \tilde{\mathbf{X}} = \mathbf{U}\tilde{\Sigma}\mathbf{V}^T \rightarrow \mathbf{G} = \mathbf{U}_k \in \mathbb{R}^{p \times k}.$$

Then the projected features are

$$\mathbf{G}^T \tilde{\mathbf{X}} = \mathbf{Z} \in \mathbb{R}^{k \times n}. \quad (3)$$

It is worth noting that the k -th row in $\mathbf{Z}$ is called the k -th principal components (PC). Therefore, PCA achieves feature reduction by keeping only the first k PCs.

Centered or not?

- ① Note that we project the centered data matrix $\tilde{\mathbf{X}}$ instead of the original data matrix $\mathbf{X}$ in Eqn (3). Maximal variance can be achieved in both cases, as the the covariance matrix $\mathbf{C}$ will not change.
- ② But the minimal reconstruction error can only be achieved when $\tilde{\mathbf{X}}$ is used, as shown in the next section.
- ③ A common practice to avoid any confusion is to center the data before applying PCA, and use the centered data matrix in all computations.

Have We Achieved the Minimal Reconstruction Error?

- 1 First, we perform the verification in the case where we project and reconstruct the centered data matrix $\tilde{\mathbf{X}}$.
- 2 Given the projection

$$\mathbf{G}^T \tilde{\mathbf{X}} = \mathbf{Z},$$

the reconstruction process is

$$\tilde{\mathbf{X}} = \mathbf{G}\mathbf{Z} = \mathbf{G}\mathbf{G}^T \tilde{\mathbf{X}},$$

and the reconstruction error is

$$\|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\|_F = \|\tilde{\mathbf{X}} - \mathbf{G}\mathbf{G}^T \tilde{\mathbf{X}}\|_F.$$

- 3 We will show that

$$\mathbf{G}\mathbf{G}^T \tilde{\mathbf{X}} = \mathbf{B}^* = \arg \min_{\mathbf{B}: \text{rank}(\mathbf{B}) \leq k} \|\tilde{\mathbf{X}} - \mathbf{B}\|_F, \quad (4)$$

which means $\tilde{\mathbf{X}}$ gives the minimum reconstruction error.

- 4 We've already learned that $\mathbf{B}^*$ is the truncated SVD of $\tilde{\mathbf{X}}$. So, we only need to show that $\mathbf{G}\mathbf{G}^T \tilde{\mathbf{X}}$ is indeed the truncated SVD of $\tilde{\mathbf{X}}$ as follows. Note that similar arguments can be made for the spectral norm.

More details

Given $\tilde{\mathbf{X}} = \mathbf{U}\tilde{\Sigma}\mathbf{V}^T$ and $\mathbf{G} = \mathbf{U}_k$, we have

$$\begin{aligned}\mathbf{G}\mathbf{G}^T\tilde{\mathbf{X}} &= \mathbf{U}_k\mathbf{U}_k^T\mathbf{U}\tilde{\Sigma}\mathbf{V}^T \\ &= \mathbf{U}_k\mathbf{U}_k^T[\mathbf{U}_k \quad \mathbf{U}_{-k}]\tilde{\Sigma}\mathbf{V}^T \\ &= \mathbf{U}_k[\mathbf{I} \quad \mathbf{0}]\tilde{\Sigma}\mathbf{V}^T \\ &= [\mathbf{U}_k \quad \mathbf{0}]\tilde{\Sigma}\mathbf{V}^T \\ &= \mathbf{U}_k\Sigma_k\mathbf{V}_k^T \\ &= \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.\end{aligned}$$

As a result, $\tilde{\mathbf{X}}$ gives the minimum reconstruction error, which is

$$\min_{\mathbf{B}: \text{rank}(\mathbf{B}) \leq k} \|\tilde{\mathbf{X}} - \mathbf{B}\|_F = \|\tilde{\mathbf{X}} - \mathbf{G}\mathbf{G}^T\tilde{\mathbf{X}}\|_F = \sqrt{\sum_{i=k+1}^n \sigma_i^2}.$$

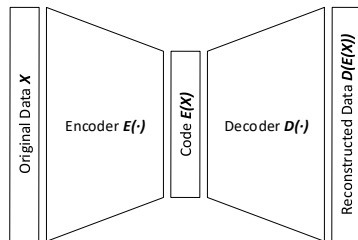
The case of spectral norm

- ① Similarly, it can be shown that

$$\min_{\mathbf{B}:\text{rank}(\mathbf{B})\leq k} \|\tilde{\mathbf{X}} - \mathbf{B}\|_2 = \|\tilde{\mathbf{X}} - \mathbf{G}\mathbf{G}^T\tilde{\mathbf{X}}\|_2 = \sigma_{k+1}.$$

- ② Therefore, PCA projects the high-dimensional features to a lower-dimensional space with minimum reconstruction error in terms of both Frobenius and spectral norms.

Connections with Autoencoders



- 1 We assume that $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \in \mathbb{R}^{p \times n}$ is already centered, for the simplicity of notations.
- 2 The outputs of this framework have the same dimension as inputs, and are supposed to reconstruct the inputs exactly. Without any constraint, the reconstruction task can be easily solved by directly copying. However, autoencoders come with a crucial restriction that the dimension of intermediate outputs should be smaller than inputs.

- 1 Concretely, the framework of autoencoders can be divided into two parts: the *encoder* $E(\cdot)$ and the *decoder* $D(\cdot)$. Given $\mathbf{X} \in \mathbb{R}^{p \times n}$ as inputs, the encoder is supposed to output a reduced representation of the inputs, *i.e.*, $E(\mathbf{X}) \in \mathbb{R}^{k \times n}$, where $k < p$. And the decoder tries to reconstruct the inputs from this reduced representation.
- 2 It is straightforward to see that the objective function of autoencoders is to minimize the reconstruction error:

$$L(\mathbf{X}, D(E(\mathbf{X}))),$$

where $L(\cdot, \cdot)$ is a loss function that measures the reconstruction error.

PCA as Autoencoders

- ① We've shown that PCA is able to achieve the minimum reconstruction error. Both PCA and autoencoders are unsupervised learning models with the goal of minimizing the reconstruction error. Are they connected?
- ② Consider a special autoencoder with the encoder and decoder defined as

$$\begin{aligned}E(\mathbf{X}) &= \mathbf{W}^T \mathbf{X} \in \mathbb{R}^{k \times n}, \\D(E(\mathbf{X})) &= \mathbf{W}E(\mathbf{X}) = \mathbf{W}\mathbf{W}^T \mathbf{X} \in \mathbb{R}^{p \times n},\end{aligned}$$

where $\mathbf{W} \in \mathbb{R}^{p \times k}$. Note that

$$\text{rank}(\mathbf{W}\mathbf{W}^T \mathbf{X}) \leq \min(\text{rank}(\mathbf{W}), \text{rank}(\mathbf{W}^T \mathbf{X})) \leq k$$

- ③ If the loss function $L(\cdot, \cdot)$ is the mean square error, the objective of this autoencoder is equivalent to

$$\min_{\mathbf{W}: \text{rank}(\mathbf{W}\mathbf{W}^T \mathbf{X}) \leq k} \|\mathbf{X} - \mathbf{W}\mathbf{W}^T \mathbf{X}\|_F. \quad (5)$$

PCA as Autoencoders

- ❶ In an autoencoder, $\mathbf{W}$ can be randomly initialized and optimized through gradient descent. However, comparing Eqn. (4) and Eqn. (5), it is easy to see that the optima can be achieved when $\mathbf{W} = \mathbf{G}$, where $\mathbf{G}$ is given by PCA.
- ❷ If the hidden units in autoencoders have linear activation functions, then it can be shown that the error function has a unique global minimum and that at this minimum the network performs a projection onto the subspace that is spanned by the principal components of the data
- ❸ Thus, the vectors of weights that lead into the hidden units in autoencoders form a basis set that spans the principal subspace. Note, however, that these vectors need not be orthogonal or normalized (Bishop DL book: Section 19.1.1)

THANKS!